1 From GPT-2 to gpt-oss: Analyzing the Architectural Advances

- **原文**

> https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the

- OpenAI 本周刚发布了两款新的开放权重的大语言模型：gpt-oss-120b 和 gpt-oss-20b，这是自 2019 年 GPT-2 以来首次推出的开放权重模型。是的，得益于一些巧妙的优化，它们可以在本地运行（稍后会详细介绍）。

  这是自 GPT-2 以来，OpenAI 首次分享体量较大的、完全开放权重的模型。早期的 GPT 模型展示了 Transformer 架构的可扩展性。2022 年的 ChatGPT 发布则通过在写作与知识（以及后来在编程）任务上的具体实用性，使这些模型走向主流。如今他们分享了一个期待已久的权重模型，其架构也包含一些有趣的细节。

  为了总结最有趣的细节，我花了过去几天的时间阅读代码和技术报告。（就在几天后，OpenAI 还发布了 GPT-5，我将在本文末尾的 gpt-oss 模型背景下简要讨论它。）

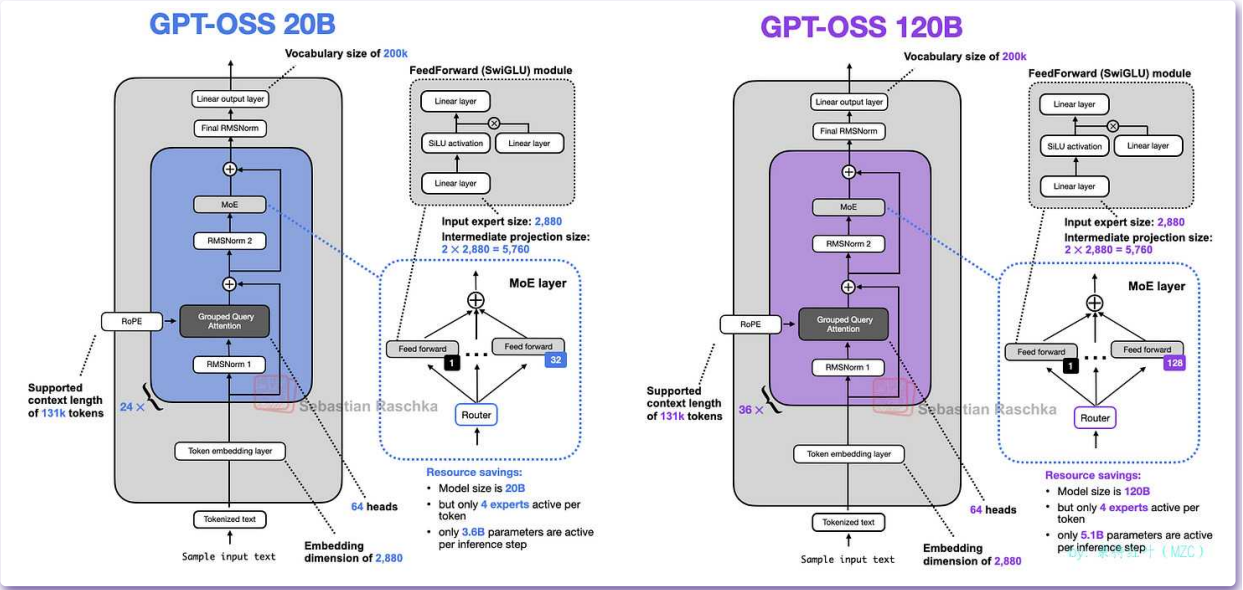
  以下是本文涵盖内容的快速预览。为了便于导航，我建议使用文章页面左侧的目录。

  1. 与 GPT-2 的模型架构比较
  2. 将 gpt-oss 模型安装到单个 GPU 上的 MXFP4 优化
  3. 宽度与深度之间的权衡 (gpt-oss vs Qwen3)
  4. 注意力偏差和 sinks
  5. 基准测试以及与 GPT-5 的比较

  希望您觉得它内容丰富！

1.1 Model Architecture Overview

- 在更详细地讨论架构之前，我们先概览一下两个模型：gpt-oss-20b 和 gpt-oss-120b，如下方图 1 所示。



如果您之前曾查看过最近的LLM架构图，或者阅读我以前的**大型体系结构比较**文章，则可能会发现乍一看没有什么新颖或不寻常的。

- 这并不令人意外，因为领先的 LLM 开发者倾向于使用相同的基本架构，然后进行较小的调整。这纯粹是我的猜测，但我认为这是因为
    1. 这些实验室之间的人才流动频繁。
    2. 我们至今仍未找到比 Transformer 架构更好的方案。尽管存在状态空间模型和文本扩散模型，但据我所知，还没有人证明它们在这一规模上能与 Transformer 相媲美。（我找到的大多数比较仅关注基准测试表现。模型在真实世界的多轮写作与编码任务中的表现仍不明确。撰写本文时，LM Arena 上排名最高的非纯 Transformer 模型是 Jamba——一种 Transformer 与状态空间模型的混合体，排名第 96 位。）
    3. 大部分性能提升很可能源于数据与算法的微调，而非重大架构变更。
- 话虽如此，他们的设计选择仍然有许多有趣的方面。上图显示了一些（而另一些则没有，但我们稍后也将讨论它们）。在本文的其余部分，我将重点介绍这些功能，并将它们一次与其他架构进行比较。

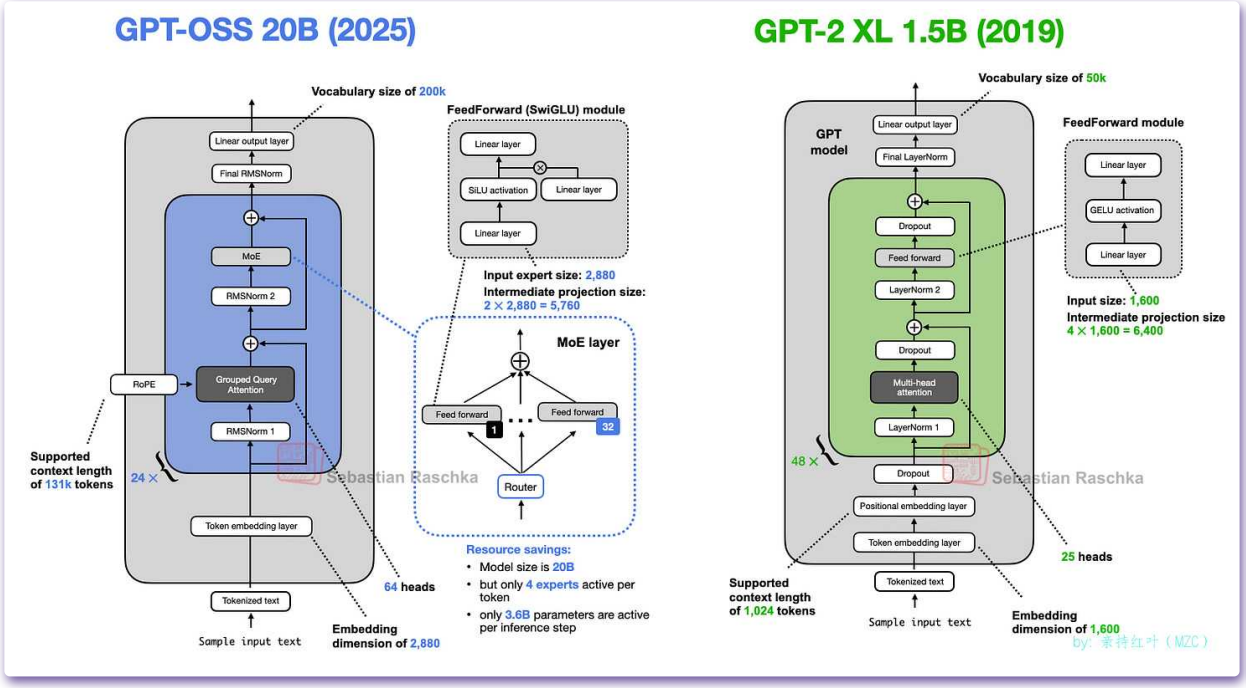
  我还要说明，我与 OpenAI 并无任何关联。我的信息来源于对已发布的模型代码的审阅以及他们技术报告的阅读。如果你想学习如何在本地使用这些模型，那最好还是阅读OpenAI官方的模型中心页面。

> https://huggingface.co/openai/gpt-oss-20b
>
> https://huggingface.co/openai/gpt-oss-120b

20B 模型可在配备最高 16 GB 显存的消费级 GPU 上运行。120B 模型可在单块配备 80 GB 显存的 H100 或更新的硬件上运行。我稍后会再讨论这一点，因为其中存在一些重要的注意事项。

1.2 Coming From GPT-2

- 在比较 gpt-oss 与更近期的架构之前，我们不妨乘坐“时间机器”，并排回看一下 GPT-2（图2），以了解技术迄今取得了多大进步。

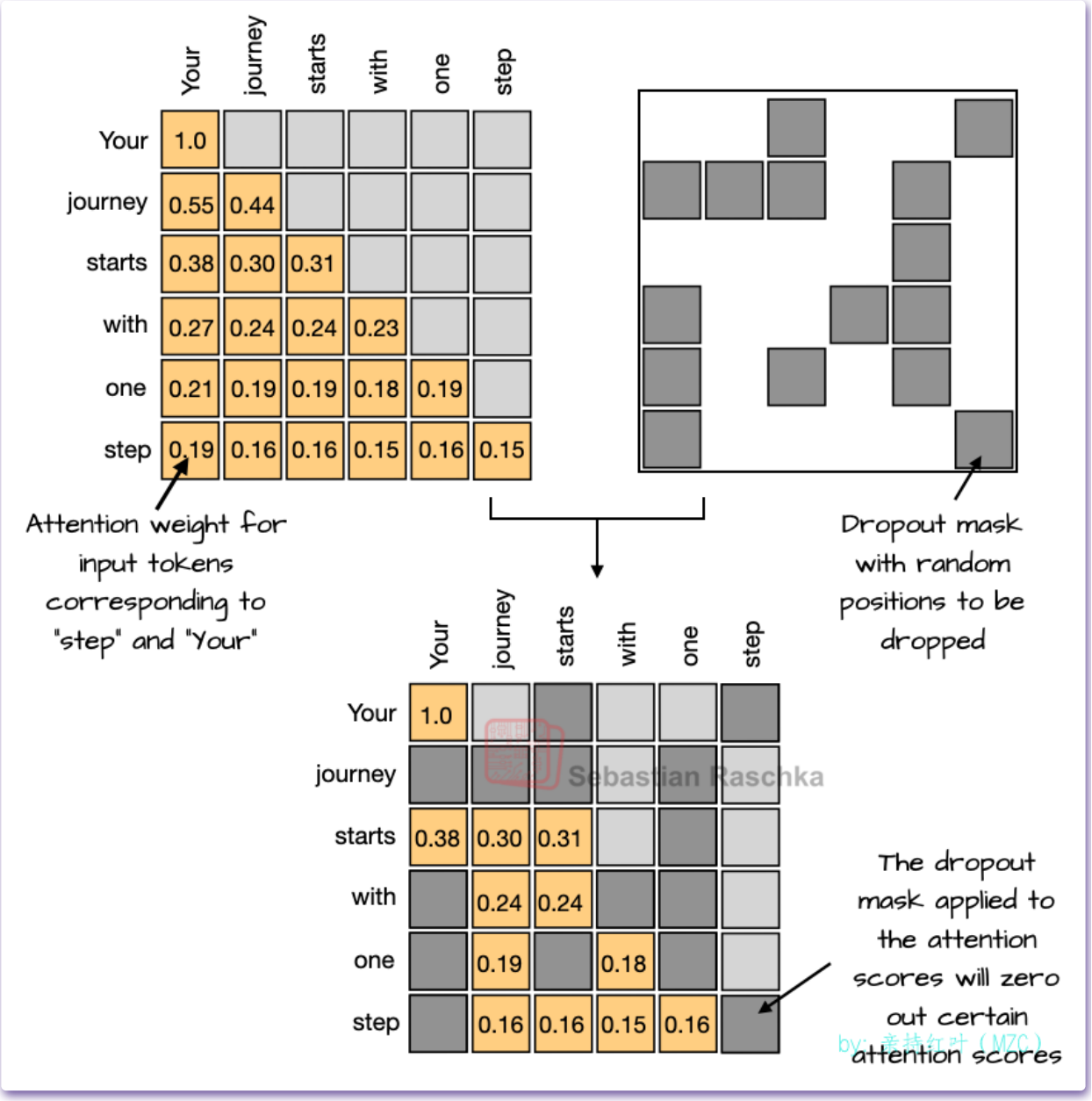


GPT-oss 和 GPT-2 都是基于 **《Attention Is All You Need》（2017 年）** 论文中提出的 Transformer 架构构建的Decode-only架构 LLM。多年来，许多细节已得到改进。

然而，这些变化并非 GPT-OSS 独有。正如我们稍后将看到的，它们也出现在许多其他 LLM 中。由于我在上一篇 **“大型架构比较”** 文章中讨论过许多此类方面，因此我将尽量使每个小节简短而有重点。

1.2.1 Removing Dropout

- Dropout（2012）是一种传统的防止过拟合技术，在训练过程中通过随机“丢弃”（即置零）一部分层激活或注意力分数（见图3）来实现。然而，在现代大语言模型中很少使用 dropout，大多数在 GPT-2 之后的模型都已将其droped（双关语并非本意）。

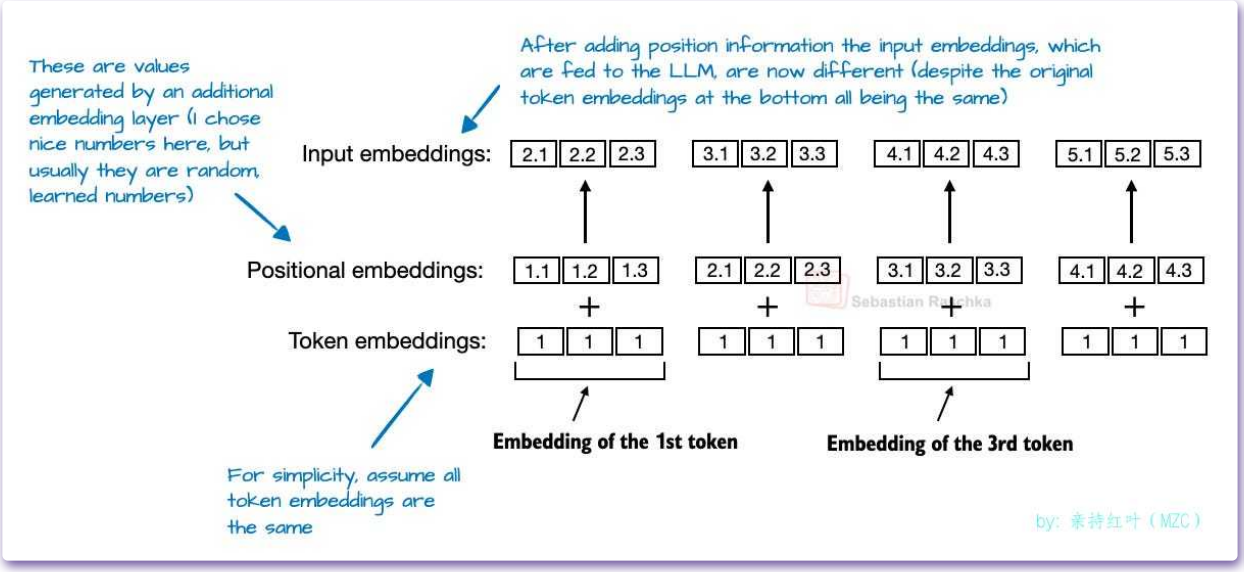


我推测 Dropout 最初在 GPT-2 中使用，是因为它继承自原始的 Transformer 架构。研究人员可能注意到，它并没有真正提升 LLM 的性能（我在小规模的 GPT-2 复制运行中也观察到了同样的情况）。这可能是因为 LLM 通常只在海量数据集上进行单轮训练，这与 Dropout 最初引入时使用的数百轮训练方案形成了鲜明对比。因此，由于 LLM 在训练过程中每个 token 只被识别一次，因此过拟合的风险很小。

有趣的是，虽然 Dropout 在 LLM 架构设计中多年来一直被忽视，但我发现了一篇 **2025 年的研究论文**，其中包含小规模 LLM 实验（Pythia 1.4B），证实了 Dropout 在这些单 epoch 机制中会导致下游性能变差。

1.2.2 RoPE Replaces Absolute Positional Embeddings

- 在基于 Transformer 的 LLM 中，由于注意力机制的存在，位置编码必不可少。默认情况下，注意力机制会将输入 token 视为无序的。在原始 GPT 架构中，绝对位置嵌入通过为序列中的每个位置添加一个学习到的嵌入向量（图 4）来解决这个问题，然后将该向量添加到 token 嵌入中。

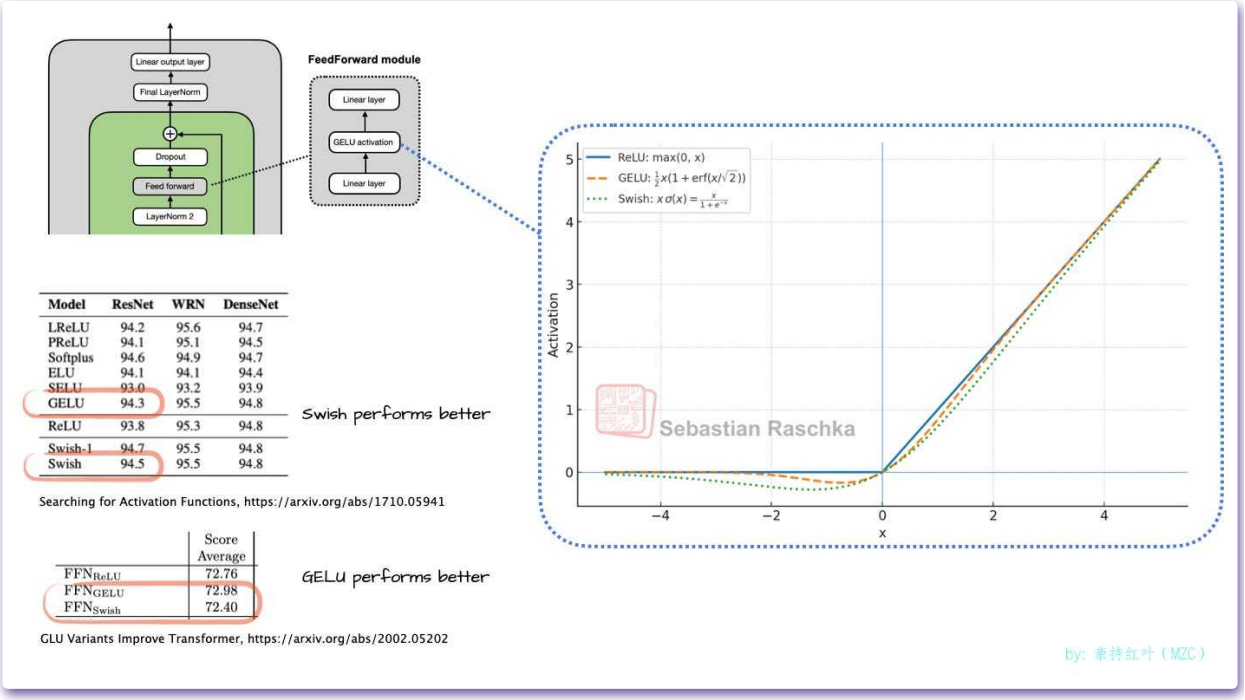


RoPE（**旋转位置嵌入**）引入了一种不同的方法：它不是将位置信息添加为单独的嵌入，而是通过根据每个标记的位置旋转查询和键向量来编码位置。（RoPE 是一个优雅的想法，但解释起来也有点棘手。我计划以后单独更详细地介绍。）

RoPE 于 2021 年首次推出，但随着 2023 年原始 Llama 模型的发布而得到广泛采用，并从此成为现代 LLM 的主要内容。

1.2.3 Swish/SwiGLU Replaces GELU

- 早期的 GPT 架构使用 GELU。为什么现在使用 Swish 而不是 GELU？我认为 Swish 的计算成本略低，这就是它的全部优势。根据你参考的论文，你会发现其中一种在建模性能方面略胜一筹。在我看来，这些细微的差异可能在标准误差范围内，具体数值会根据超参数敏感度而有所不同。
- 激活函数曾经是一个热门的争论话题，直到十多年前深度学习社区基本确定了 ReLU 函数。从那时起，研究人员提出并尝试了许多类似 ReLU 且曲线更平滑的变体，其中 GELU 和 Swish（图 5）是其中最受青睐的。



Model	ResNet	WRN	DenseNet
LReLU	94.2	95.6	94.7
PReLU	94.1	95.1	94.5
Softplus	94.6	94.9	94.7
ELU	94.1	94.1	94.4
SELU	93.0	93.2	93.9
GELU	94.3	95.5	94.8
ReLU	93.8	95.3	94.8
Swish-1	94.7	95.5	94.8
Swish	94.5	95.5	94.8

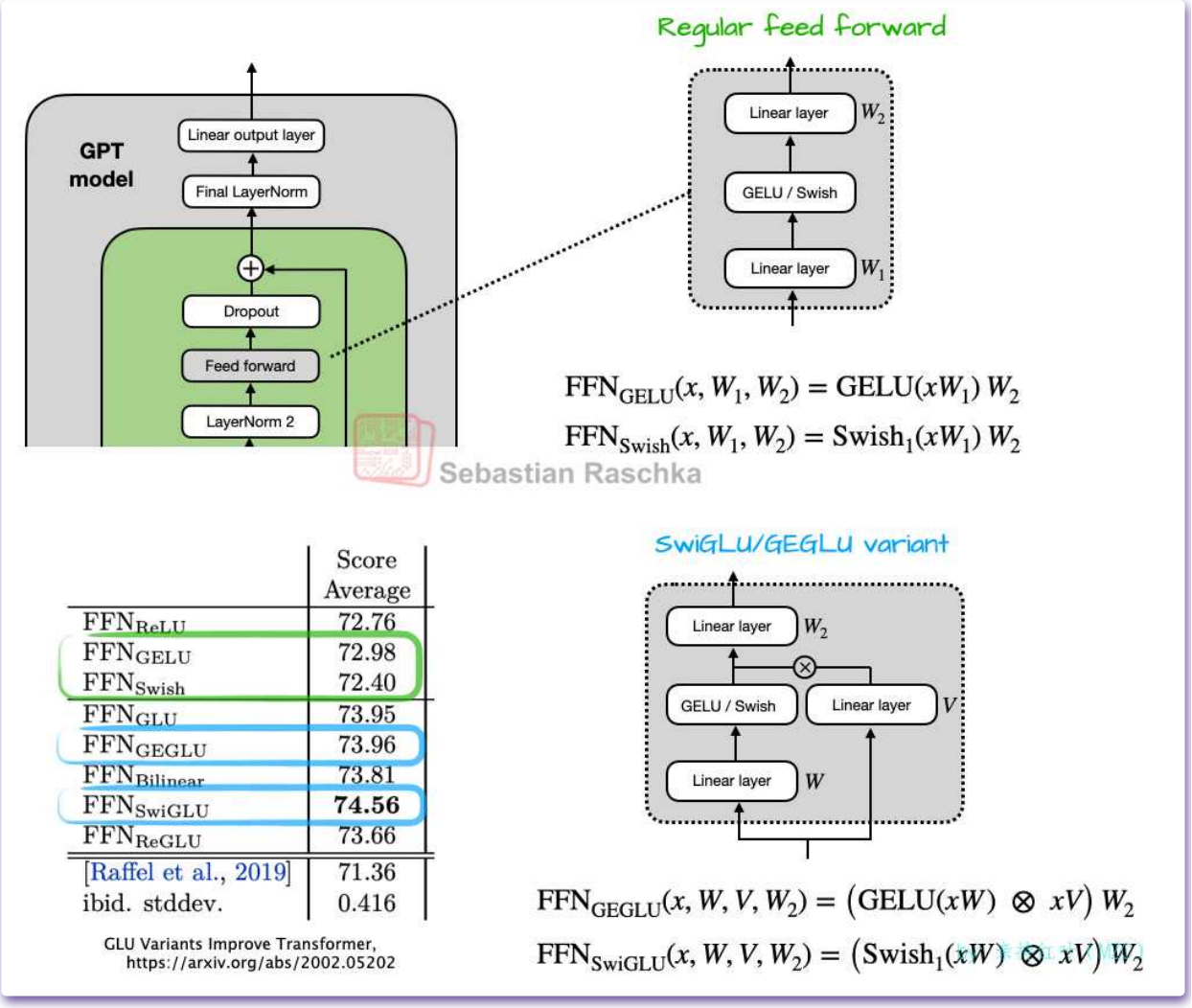
	Score Average
FFN_{ReLU}	72.76
FFN_{GELU}	72.98
FFN_{Swish}	72.40

早期的 GPT 架构使用 GELU，其定义为 `0.5x * [1 + erf(x / sqrt(2))]` 。其中， `erf` （误差函数的缩写）是高斯积分，它是使用高斯积分的多项式近似来计算的，这使得它的计算成本比 Swish 中使用的 S 形函数（其中 Swish 只是 `x * sigmoid(x)` 简单函数更高 。

实际上，Swish 的计算成本略低于 GELU，这可能是它在大多数新模型中取代 GELU 的主要原因。根据我们所参考的论文，其中一种算法的建模性能可能会略胜一筹。但我认为这些改进通常在标准误差范围内，最终胜出者将很大程度上取决于超参数的调优。

- 如今，大多数架构都使用 Swish。然而，GELU 并未被完全遗忘；例如，谷歌的 Gemma 模型仍然使用 GELU。

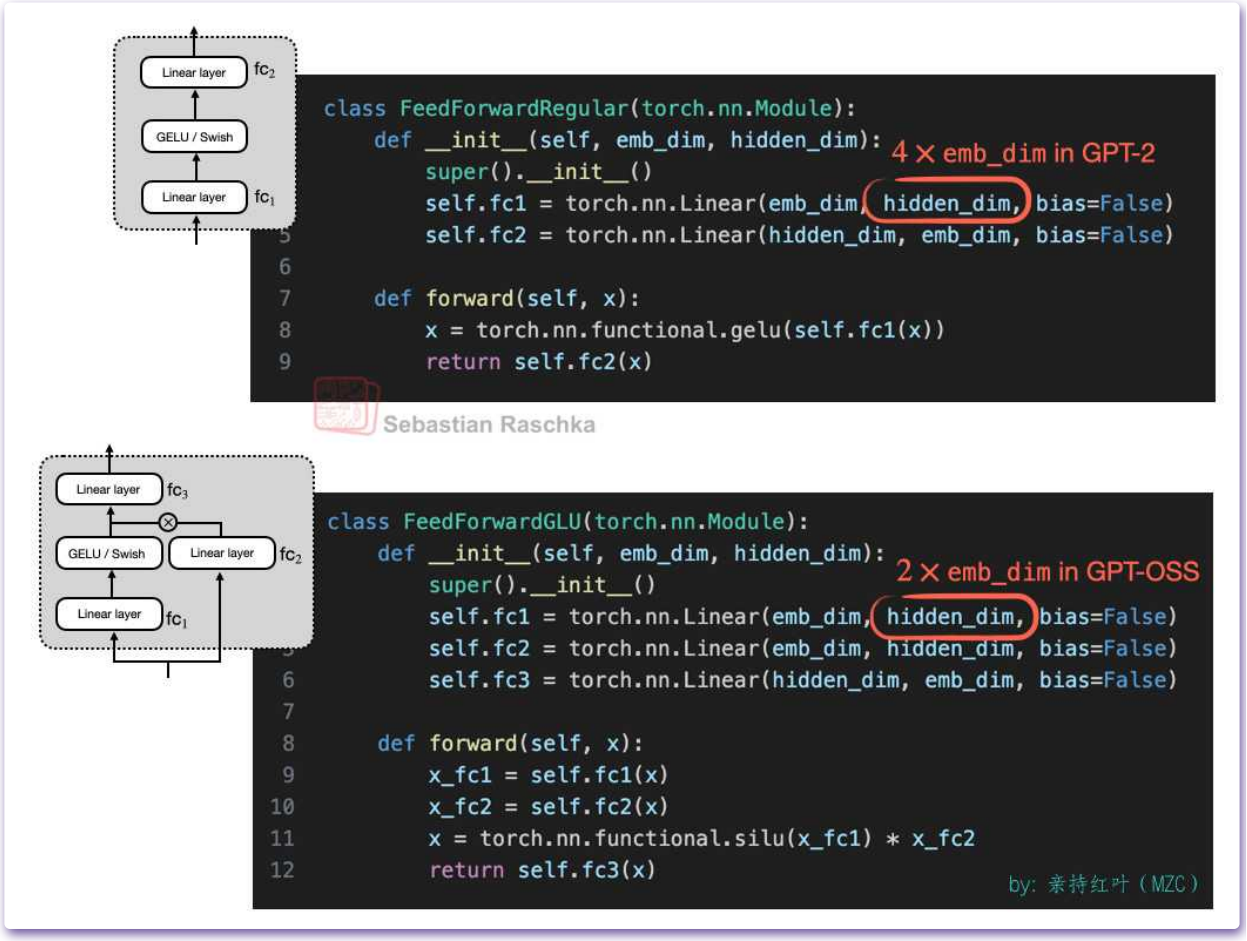
  更值得注意的是，FFN模块（一个小型MLP）被门控“GLU”模块取代。GLU 是门控线性单元的缩写，是在 **2020 年的一篇论文** 中提出的 。具体来说，2 个全连接层被 3 个全连接层取代，如下图 6 所示。



	Score Average
$\mathrm{FFN}_{\mathrm{ReLU}}$	72.76
$\mathrm{FFN}_{\mathrm{GELU}}$	72.98
$\mathrm{FFN}_{\mathrm{Swish}}$	72.40
$\mathrm{FFN}_{\mathrm{GLU}}$	73.95
$\mathrm{FFN}_{\mathrm{GEGLU}}$	73.96
$\mathrm{FFN}_{\mathrm{Bilinear}}$	73.81
$\mathrm{FFN}_{\mathrm{SwiGLU}}$	**74.56**
$\mathrm{FFN}_{\mathrm{ReGLU}}$	73.66
[Raffel et al., 2019]	71.36
ibid. stddev.	0.416

乍一看，GEGLU/SwiGLU 变体似乎比常规前馈层更好，因为额外的层带来了更多的参数。但这并非易事，因为在实践中， SwiGLU/GEGLU 中的 `W` 和 `V` 权重层通常被选择为传统前馈层中 `W_1` 层大小的一半 。

为了更好地说明这一点，请考虑常规和 GLU 变体的具体代码实现：



因此，假设我们的嵌入维度为 1024。在常规前馈情况下，这将是

$$
\begin{aligned}
fc1 &: 1024 * 4096 = 4194304 \\
fc2 &: 1024 * 4096 = 4194304
\end{aligned}
\tag{1}
$$

即 $fc1 + fc2 = 8388608$ 个参数

对于 GLU 变体，我们有

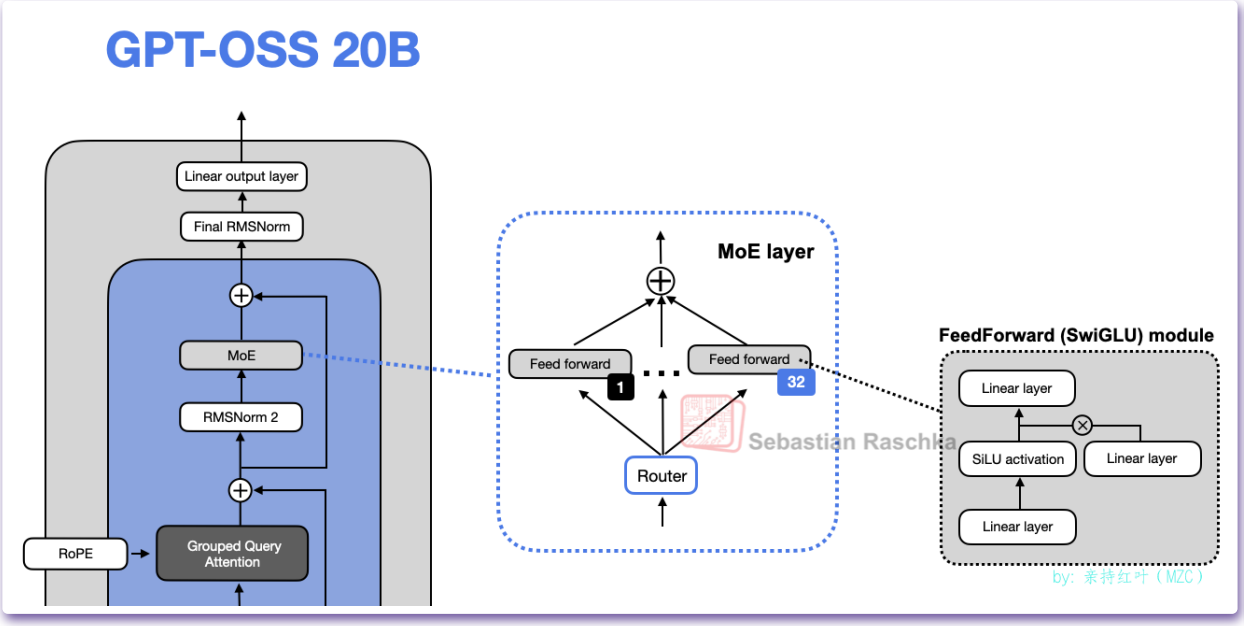
$$
\begin{aligned}
fc1 &: 1024 * 2048 = 2097152 \\
fc2 &: 1024 * 2048 = 2097152 \\
fc3 &: 2048 * 1024 = 2097152
\end{aligned}
\tag{2}
$$

即 $3 * 2097152 = 6291456$ 个参数

总体而言，采用 GLU 变体可以减少参数数量，同时获得更好的性能。其性能提升的原因在于，这些 GLU 变体引入了额外的乘性交互，从而提升了表达能力（与深而窄的神经网络在良好训练条件下优于浅而宽的神经网络的原因相同）。

1.2.4 Mixture-of-Experts Replaces Single FeedForward Module

- 除了将FFN模块升级为 SwiGLU（如上文所述）之外，gpt-oss 还将单个FFN模块替换为多个FFN模块，每个 token 生成步骤仅使用一个子集。这种方法被称为混合专家 (MoE)，如下图 8 所示。



因此， 用多个前馈模块替换单个前馈模块 （就像在 MoE 设置中所做的那样）会显著增加模型的总参数数量。然而，关键在于我们不会为每个 token 使用（“激活”）所有专家。相反，路由器只会为每个 token 选择一小部分专家。

- 由于每次仅有少数专家处于激活状态，MoE 模块常被称为稀疏模块，与始终使用完整参数集的致密模块相对。借助 MoE 带来的庞大参数总量，LLM 的容量得以提升，即在训练中能吸收更多知识。然而，由于稀疏性在推理阶段无需同时调用全部参数，因此推理仍然高效。

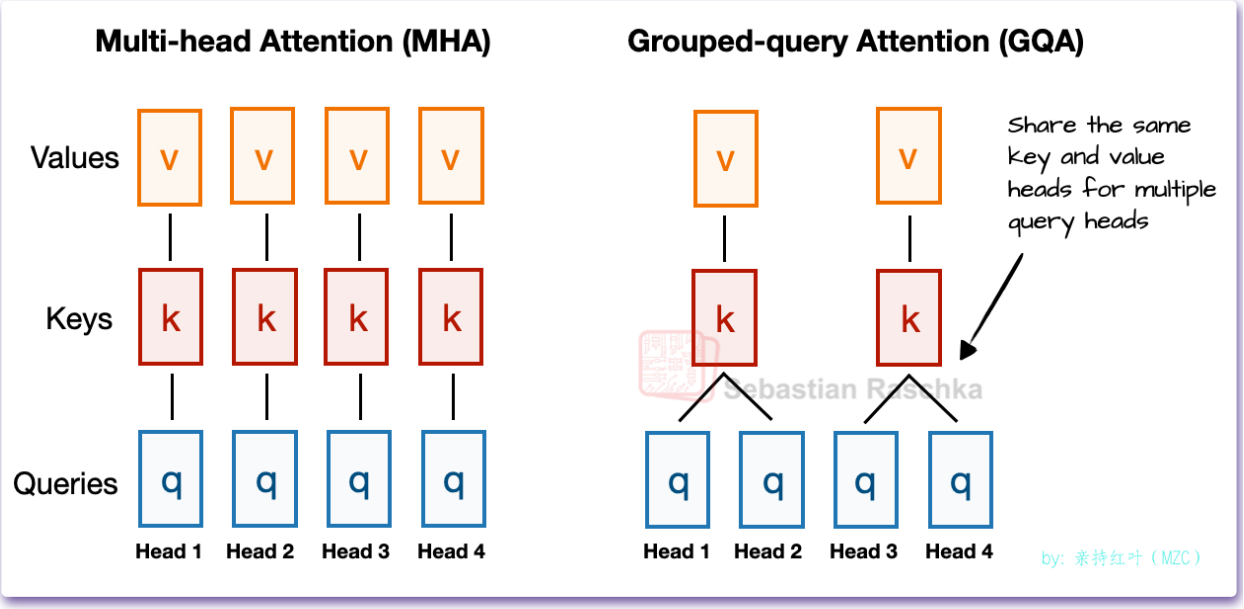
  （有趣的是：在大多数 MoE 模型中，专家权重占模型总参数的 90% 以上。）

1.2.5 Grouped Query Attention Replaces Multi-Head Attention

- 正如我在之前的文章中提到的，分组查询注意力（GQA）近年来已成为多头注意力（MHA）更高效的计算和参数替代方案。

  在 MHA 中，每个 head 都有自己的一组键和值。GQA 通过将多个 head 分组以共享相同的键和值投影来减少内存使用量。

  例如，如图 9 所示，如果有 2 个键值组和 4 个注意力头，则注意力头 1 和 2 可能共享一组键值对，而注意力头 3 和 4 则共享另一组键值对。这种分组减少了键值对的计算总量，从而降低了内存使用量并提高了效率——根据消融研究，这不会显著影响建模性能。



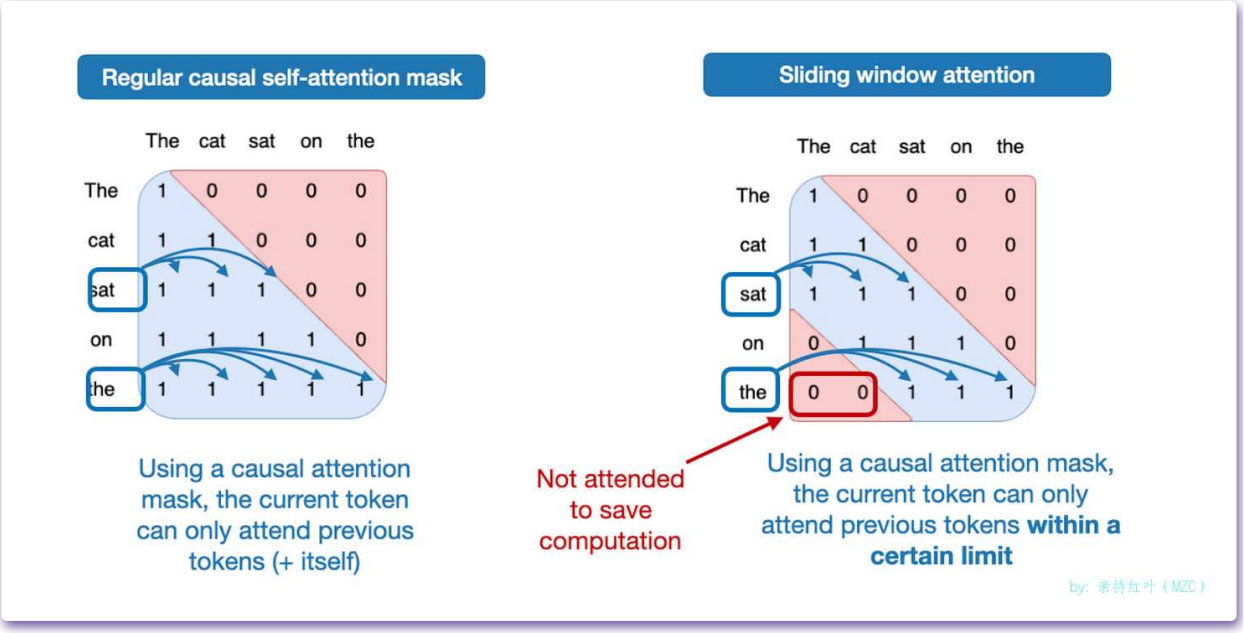
因此，GQA 的核心思想是通过在多个查询头之间共享键值头来减少键值头的数量。这 (1) 可以降低模型的参数数量，(2) 还可以减少推理过程中键值张量的内存带宽占用，因为需要存储和从键值缓存中检索的键值更少。

（如果您好奇 GQA 在代码中的样子，请参阅我的 **GPT-2 到 Llama 3 转换指南** ， **其中** 有一个没有 KV 缓存的版本，以及一个 KV 缓存变体 。）

- 尽管 GQA 主要是为多头注意力（MHA）提供计算效率上的权衡方案，但消融研究（例如原始 GQA 论文和 Llama 2 论文中的研究）表明，在大型语言模型的建模性能方面，它与标准 MHA 表现相当。

1.2.6 Sliding Window Attention

- 滑动窗口注意力机制（如下图 10 所示）最早在 **LongFormer 论文（2020 年）** 中提出，后来由 Mistral 推广。有趣的是，GPT-OSS 每隔一层就应用一次滑动窗口注意力机制。你可以将其视为多头注意力机制（在本例中为分组查询注意力机制 (GQA)）的一种变体，其中注意力上下文被限制在一个较小的窗口中，从而同时降低了内存使用量和计算成本。



具体而言，gpt-oss 在 GQA 层之间交替使用两种机制：一种对完整上下文进行注意力计

算，另一种采用仅限于 128 个标记的滑动窗口注意力。

正如我在 **上一篇文章** 中所讨论的， **Gemma 2 (2024)** 采用了类似的 1:1 比例。 今年早些时候发布的 **Gemma 3** 则更进一步，改为 5:1 的比例，这意味着每五个滑动窗口（局部）注意力层对应一个全注意力层。

- 根据 Gemma 的消融研究，滑动窗口注意力机制对建模性能的影响微乎其微，如下图所示。需要注意的是，Gemma 2 中的窗口大小为 4096 个 token，而 Gemma 3 中将其缩减至 1024 个。在 GPT-OSS 中，窗口大小仅为 128 个 token，非常小。
  有趣的是， **官方公告文章** 指出，滑动窗口注意力显然已经在 GPT-3 中使用：

> The models use alternating dense and locally banded sparse attention patterns, similar to GPT-3

谁知道呢！我回去看了 **GPT-3 的原始论文** ，那里确实提到了这一点：

> We use the same model and architecture as GPT-2 [RWC+19], including the modified initialization, pre-normalization, and reversible tokenization described therein, with the exception that we use alternating dense and locally banded sparse attention patterns in the layers of the transformer, similar to the Sparse Transformer

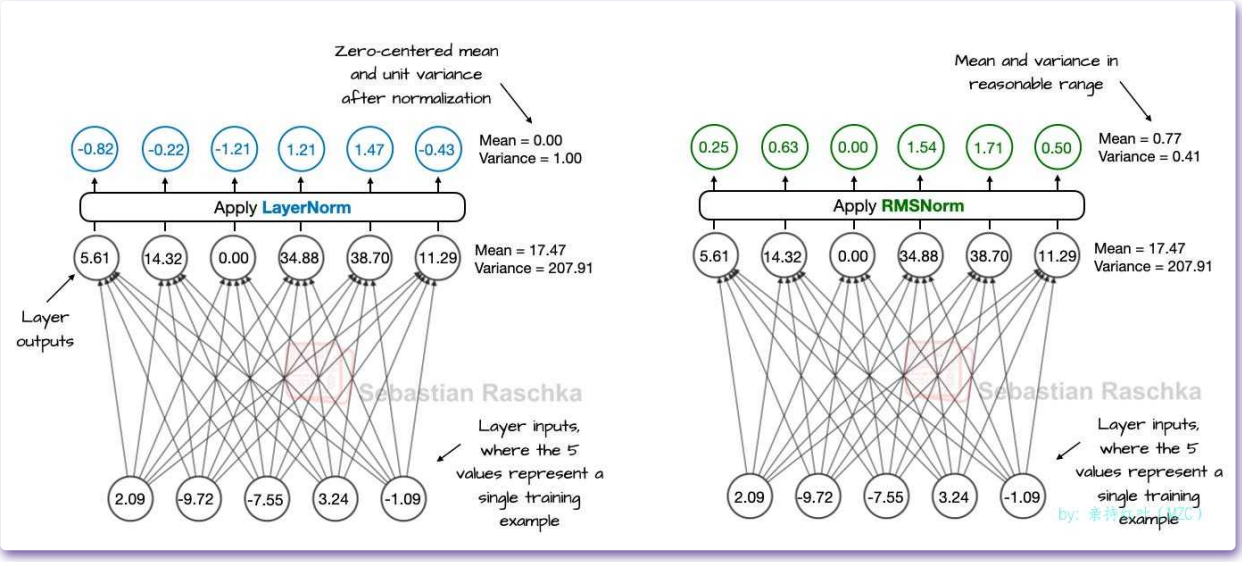
---

1.2.7 RMSNorm Replaces LayerNorm

- 最后，GPT-2 做出的最后一个小调整是用 **RMSNorm (2019)** 取代了 **LayerNorm (2016)** ，这是近年来的普遍趋势。

  类似于用 Swish 和 SwiGLU 替换 GELU，RMSNorm 是这些较小但有效的效率改进之一。RMSNorm 与 LayerNorm 类似，其目的都是对层激活进行归一化，如下图所示。

  你或许还记得，不久之前 BatchNorm 还是完成这项任务的首选。此后它逐渐失宠，主要原因在于其均值与方差等批量统计量使高效并行化更困难，而且在小批量规模下表现不佳。



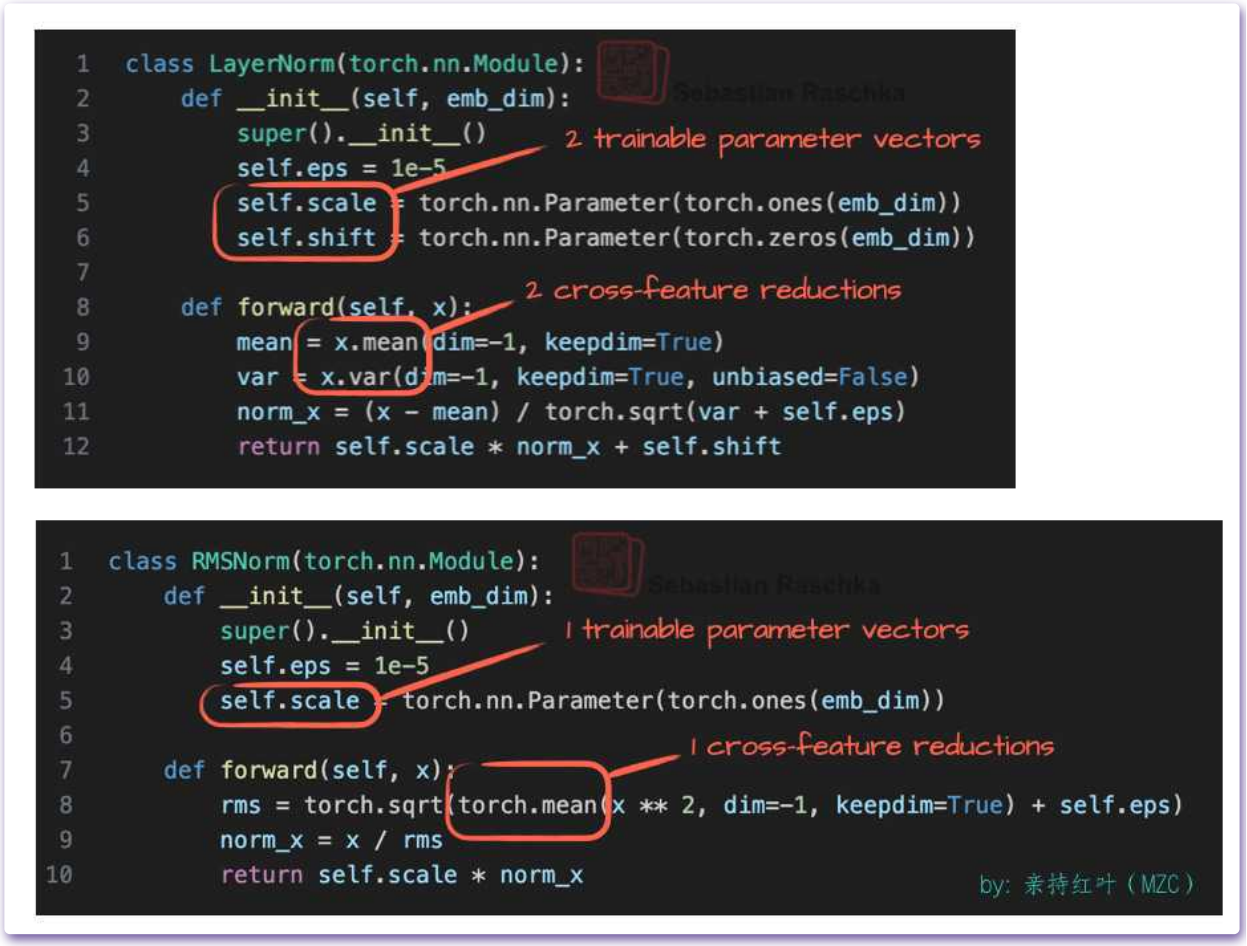
如上所示，LayerNorm 与 RMSNorm 都会将层的输出缩放至合理范围内。

LayerNorm 减去平均值并除以标准差，使得层输出具有零平均值和单位方差（方差为 1，标准差为 1）。

RMSNorm 将输入除以均方根。这不会强制要求均值为零且方差为单位，但均值和方差应处于合理范围内：均值为 -1 到 1，方差为 0 到 1。在上 所示的特定示例中，均值为 0.77，方差为 0.41。

LayerNorm 和 RMSNorm 都能稳定激活的尺度并改进优化过程，但在大规模 LLM 中常优先采用 RMSNorm，因为其计算成本更低。与 LayerNorm 不同，RMSNorm 不包含偏置（平移）项，并将昂贵的均值与方差计算简化为一次均方根运算。这样将跨特征的归约次数从两次减少为一次，降低了 GPU 上的通信开销，提高了训练效率。

下图显示了其代码



1.2.8 The GPT-2 Legacy

- 我仍然认为，在学习 LLM 时，GPT-2 是一个优秀的入门架构。它足够简单易懂，不会迷失在层层优化技巧中，但又足够复杂，能够让你扎实掌握现代 Transformer 模型的工作原理。

  从 GPT-2 开始，您可以专注于基础知识（注意力机制、位置嵌入、规范化和整体训练流程），而不会被新架构中的额外功能和调整所淹没。

  事实上，我认为在尝试叠加较新的改动之前，先花时间学习乃至实现 GPT-2 是值得的。这样不仅更容易理解那些改动，你也更可能更好地欣赏它们，因为你会更清楚这些改动试图解决的局限或问题。

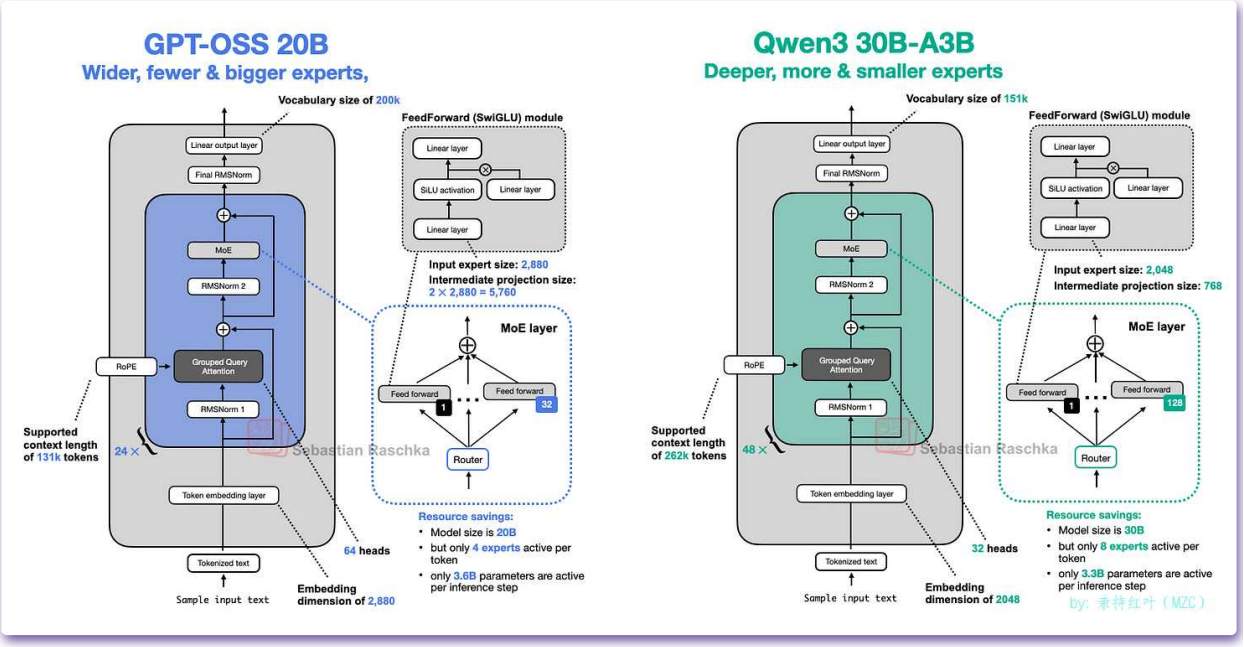
  例如，从我的 GPT-2 代码开始，我最近 **从头开始实现了 Qwen3 架构** ，它与 gpt-oss 非常相似，这将我们带到下一个主题：将 gpt-oss 与更新的架构进行比较。

1.3 Comparing gpt-oss To A Recent Architecture (Qwen3)

- 现在我们已经了解了从 GPT-2 到 GPT OSS 的演变过程，我们可以进行下一步，将 GPT OSS 与三个月前（2025 年 5 月）发布的较新的架构 Qwen3 进行比较。

  我之所以选择 Qwen3，是因为截至撰写本文时，它是顶级的开放权重模型之一。此外，Qwen3 的 MoE 模型之一，由于其可训练参数的整体规模相对接近，几乎可以直接与 GPT OSS 进行比较。

  下图 13 将 gpt-oss-20b 与大小相当的 Qwen3 模型进行了比较。

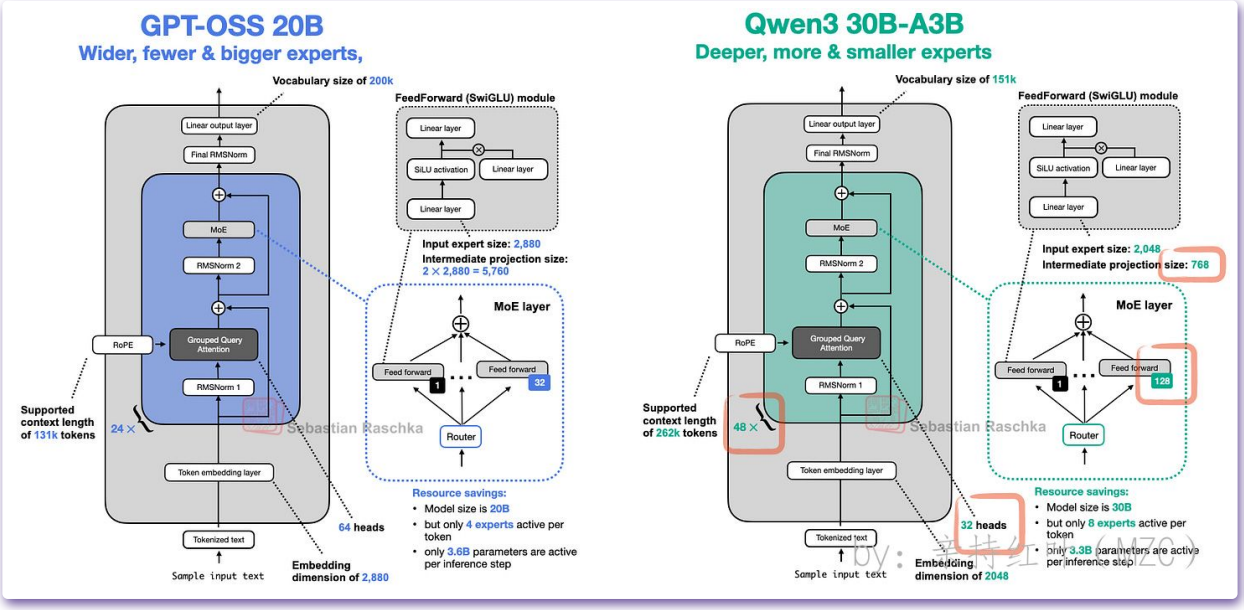


  我们可以看到，GPT-OSS 20B 和 Qwen3 30B-A3B 在架构组件上非常相似。除了尺寸之外，主要的区别在于 GPT-OSS 采用了滑动窗口注意力机制（如前文 1.6 节所述）（图中未显示），而 Qwen3 则没有。

  在下面的小节中，我们将逐一介绍值得注意的细节。

1.3.1 Width Versus Depth

- 如果我们仔细观察这两个模型，我们会发现 Qwen3 是一个更深的架构，它有 48 个变压器块而不是 24 个



另一方面，gpt-oss 是一个更宽的架构

- embedding维度为 2880，而不是 2048
- 专家（FFN）投影维度为 5760，而不是 768

值得注意的是，gpt-oss 使用了两倍的注意力头，但这并不会直接增加模型的宽度。宽度由嵌入维度决定。

- 在参数数量固定的情况下，这两种方法是否各有优势？一般而言，更深的模型更具灵活性，但由于梯度爆炸和梯度消失等导致的不稳定问题，训练会更困难（RMSNorm 和残差连接旨在缓解这些问题）。

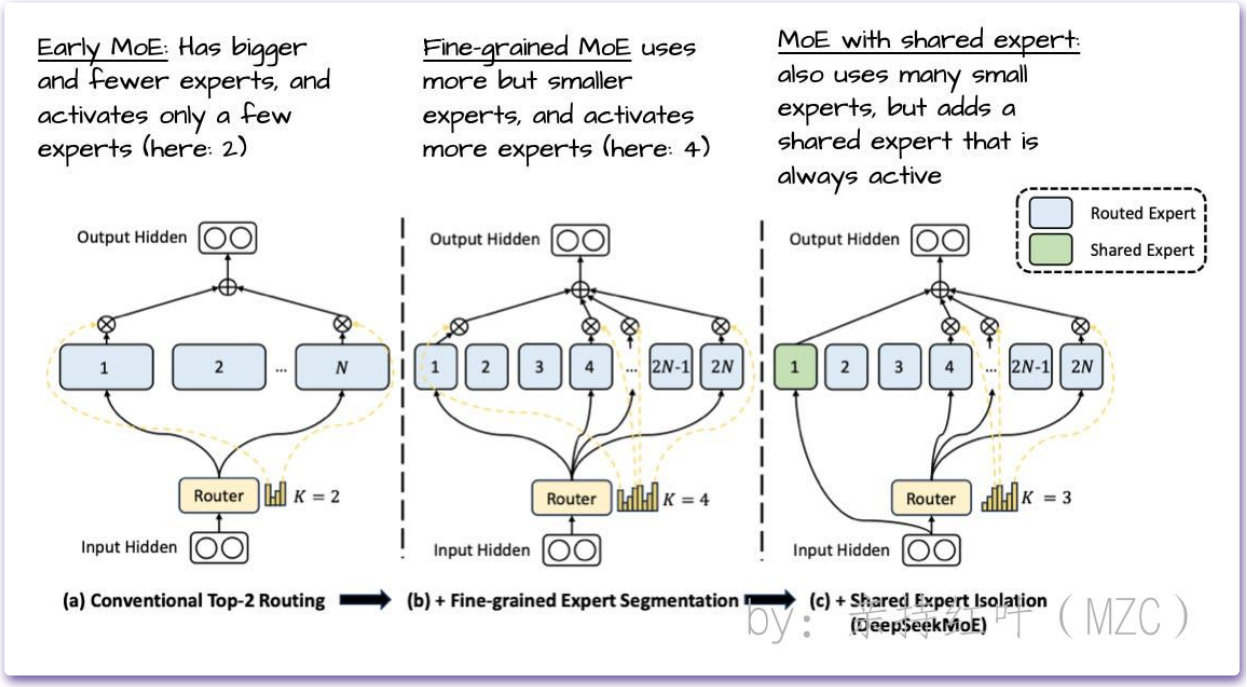
  更宽的架构具有推理速度更快的优势（具有更高的tokens/s吞吐量，tps），因为可以在更高的内存成本下实现更好的并行化。

  在模型性能方面，除了一项来自《Gemma 2》论文（表 9）的消融研究外，我所知并没有在参数规模和数据集保持一致条件下的、可真正对等比较的结果。该研究发现，对于一个 90 亿参数的架构，宽度更大的设计略优于深度更深的设计。在四个基准上，宽模型的平均得分为 52.0，深模型的平均得分为 50.8。

1.3.2 Few Large Versus Many Small Experts

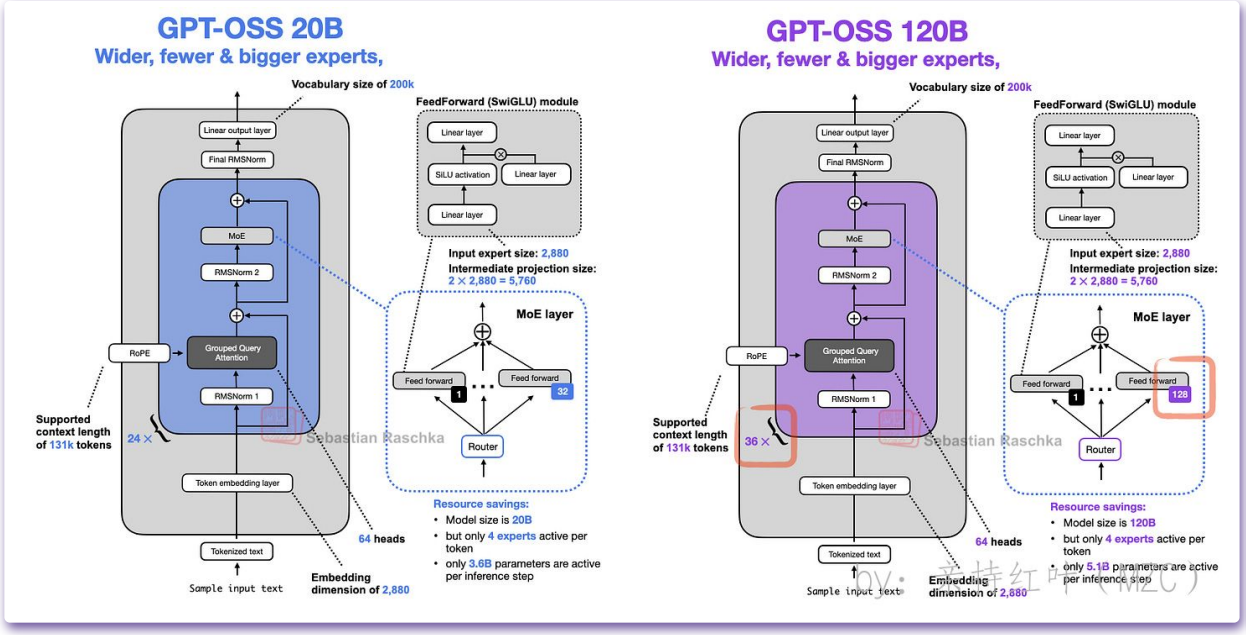
- 如上图所示，还有件值得注意的是gpt-oss 的专家数量出奇地少（32 个而不是 128 个），并且每个 token 仅激活 4 个而不是 8 个专家。然而，每个专家的数量比 Qwen3 中的专家数量要大得多。

  这很有意思，因为最近的趋势和发展表明，更多、更小的模型是有益的。在总参数大小不变的情况下，这种变化在 DeepSeekMoE 论文的图 15 中得到了很好的说明。



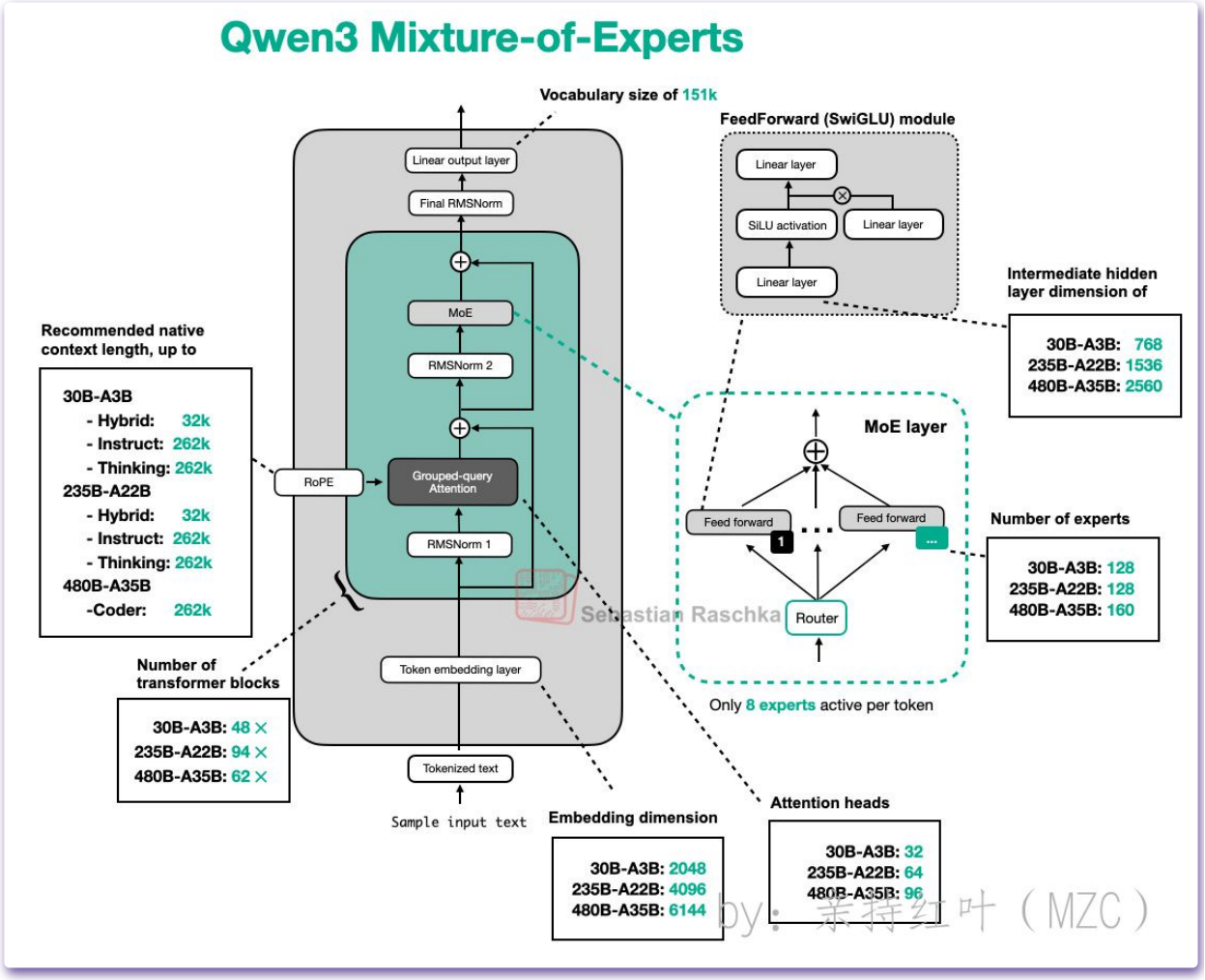
需要注意，与 DeepSeek 的模型不同，gpt-oss 和 Qwen3 都没有使用共享专家。

- 客观而言，gpt-oss 中专家数量偏少，可能是 20B 规模带来的副作用。观察下面的 120B 模式，他们确实在保持其他条件不变的情况下，增加了专家数量（以及 Transformer 块数），如下方图所示。



对于 20B 和 120B 模型如此相似这一现象，一个枯燥的解释可能是 120B 模型是主要的研究重点。 创建较小模型的简单方法是使其短一些（减少 Transformer 块的数量）并减少专家数量，因为大部分参数都在那里。 然而，人们可能会猜测，他们是否先开始训练 120B 模型，然后为了继续预训练而删除了一些 Transformer 块和专家（而不是从随机初始权重开始）。

- 无论如何，这是因为仅缩放这两个（Transformer 模块和专家数量）的情况相当罕见。例如，当查看多种尺寸的 Qwen3 MoE 模型（下图）时，它们在更多方面彼此之间进行了更均衡的缩放。



1.3.3 Attention Bias and Attention Sinks

- 两者都用了**分组查询注意力（GQA）**，但区别在于 **gpt-oss** 还在网络里隔一层就用一次**滑动窗口注意力**来限制每层可看的上下文范围，而 **Qwen3** 没有这种“隔层做局部注意力”的做法。

  两者都用了分组查询注意力。不同之处在于，gpt-oss 在注意力权重里加入了可学习的偏置单元（在softmax前给每个头附加偏置项），而这点在图里也有体现

```python
class GptOssAttention(Qwen2Attention):
    def __init__(self, config: GptOssConfig, layer_idx: int):
        super().__init__(config, layer_idx)
        self.q_proj = nn.Linear(
            config.hidden_size, config.num_attention_heads * self.head_dim, bias=config.attention_bias
        )
        self.k_proj = nn.Linear(
            config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
        )
        self.v_proj = nn.Linear(
            config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
        )
        self.o_proj = nn.Linear(
            config.num_attention_heads * self.head_dim, config.hidden_size, bias=config.attention_bias
        )
        self.sinks = nn.Parameter(torch.empty(config.num_attention_heads))
```

```json
{
  "architectures": [
    "GptOssForCausalLM"
  ],
  "attention_bias": true,
  "attention_dropout": 0.0,
  "eos_token_id": 200002,
  "experts_per_token": 4,
  "head_dim": 64,
  "hidden_act": "silu",
  "hidden_size": 2880,
  "initial_context_length": 4096,
  "initializer_range": 0.02,
  "intermediate_size": 2880,
```

by：柔特红叶（MZC）

我从 GPT-2 之后也很少再看到这些偏置单元被使用，而且它们通常被认为是多余的。确实，我找到了一篇近期论文，数学上证明至少对键变换（k_proj）来说这是成立的。并且，实证结果也显示带不带偏置单元的差别很小（见下方图）

	GPT-2	RoBERTa-base
Original	2.9251	5.8890
No b_k	2.9250	5.8909

Test set loss; lower is better

还有一个你可能注意到的细节：上图代码截图里对 “sinks” 的定义。一般的模型里，attention sinks 指的是放在序列开头、总是会被关注的特殊标记，用来稳定注意力，尤其在长上下文场景里很有用。也就是说，当上下文变得很长时，开头这个特别被关注的标记仍然会被注意到，并且它可以学会存储对整段序列都普遍有用的信息。（我认为这个做法最早来自这篇论文：Efficient Streaming Language Models with Attention Sinks 。）

在 gpt-oss 的实现里，attention sinks 并不是输入序列里的真实标记；相反，它们是为每个注意力头单独学习的偏置 logits，被直接加到注意力分数上（见下图）。这样做的目的和前面提到的 attention sinks 一样，但不需要改动分词后的输入。

```
228    class GptOssAttention(Qwen2Attention):
229        def __init__(self, config: GptOssConfig, layer_idx: int):
230            super().__init__(config, layer_idx)
231            self.q_proj = nn.Linear(
232                config.hidden_size, config.num_attention_heads * self.head_dim, bias=config.attention_bias
233            )
234            self.k_proj = nn.Linear(
235                config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
236            )
237            self.v_proj = nn.Linear(
238                config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
239            )
240            self.o_proj = nn.Linear(
241                config.num_attention_heads * self.head_dim, config.hidden_size, bias=config.attention_bias
242            )
243            self.sinks = nn.Parameter(torch.empty(config.num_attention_heads))

...

206        key_states = repeat_kv(key, module.num_key_value_groups)
207        value_states = repeat_kv(value, module.num_key_value_groups)
208        attn_weights = torch.matmul(query, key_states.transpose(2, 3)) * scaling
209        if attention_mask is not None:
210            causal_mask = attention_mask[:, :, :, : key_states.shape[-2]]
211            attn_weights = attn_weights + causal_mask
212
213        sinks = module.sinks.reshape(1, -1, 1, 1).expand(query.shape[0], -1, query.shape[-2], -1)
214        combined_logits = torch.cat([attn_weights, sinks], dim=-1)
```

by：柔持红叶（MZC）

1.3.4 License

- 最后，与 Qwen3 类似，gpt-oss 模型采用 Apache 2.0 开源许可证，这很棒（我自己的开源项目也倾向于使用相同的许可证）。这意味着这些模型可以不受限制地提炼成其他模型或用于商业产品。
- **开放权重与开源 LLM。** 多年来，这种区别一直备受争议，但为了避免与本次发布及其相关

构件产生混淆，有必要进行澄清。一些模型开发者仅发布模型权重和推理代码（例如 Llama、Gemma、gpt-oss），而其他一些模型开发者（例如 OLMo）则将所有内容（包括训练代码、数据集和权重）以真正的开源形式发布。

- 按照更严格的定义，gpt-oss 是一个 *开放权重* 模型（就像 Qwen3 一样），因为它包含权重和推理代码，但不包含训练代码或数据集。然而，业界对该术语的使用并不一致。

  我假设“gpt-oss”中的“oss”代表 *开源软件* ；然而，令我惊喜的是，OpenAI 本身在其官方 **公告文章** 中明确将 gpt-oss 描述为开放权重模型 。

1.4 Other Interesting Tidbits

- 虽然前面几节描述了该架构自 GPT-2 以来的演变，并讨论了它与 Qwen3（以及大多数其他近期模型）的相似之处，但还有一些值得注意的细节我尚未提及。这些要点与前面几节的内容并不完全契合，但仍然值得一提。

1.4.1 Training Overview

- 遗憾的是，关于训练集大小和算法的信息并不多。我从 **模型卡报告** (1) 和 **公告帖** (2) 中整理了一些最有趣的信息，如下所示：

> The gpt-oss models were trained using our most advanced pre-training and post-training techniques [...] (1)
> gpt-oss 模型采用我们最先进的预训练和后训练技术进行训练 [...] (1)

> [...] required 2.1million H100-hours to complete, with gpt-oss-20b needing almost 10x fewer. (1)
> [...] 需要 210 万个 H100 小时才能完成，而 gpt-oss-20b 所需的时间几乎减少了 10 倍。(1)

> [...] including a supervised fine-tuning stage and a high-compute RL stage [...] (2)
> [...] 包括一个监督微调阶段和一个高计算 RL 阶段 [...] (2)

> We trained the models on a mostly English, text-only dataset, with a focus on STEM, coding, and general knowledge. (2)
> 我们在一个主要由英语和纯文本组成的数据集上训练了模型，重点关注 STEM、编码和常识。(2)

因此，我们知道 gpt-oss 模型是推理模型。210 万个 H100 GPU 小时的训练计算量大致相当于训练规模约 5.6 倍的 **DeepSeek V3** 模型所需的 278.8 万个 H800 GPU 小时。遗憾的是，目前还没有关于 Qwen3 训练时间的信息可以来对比。

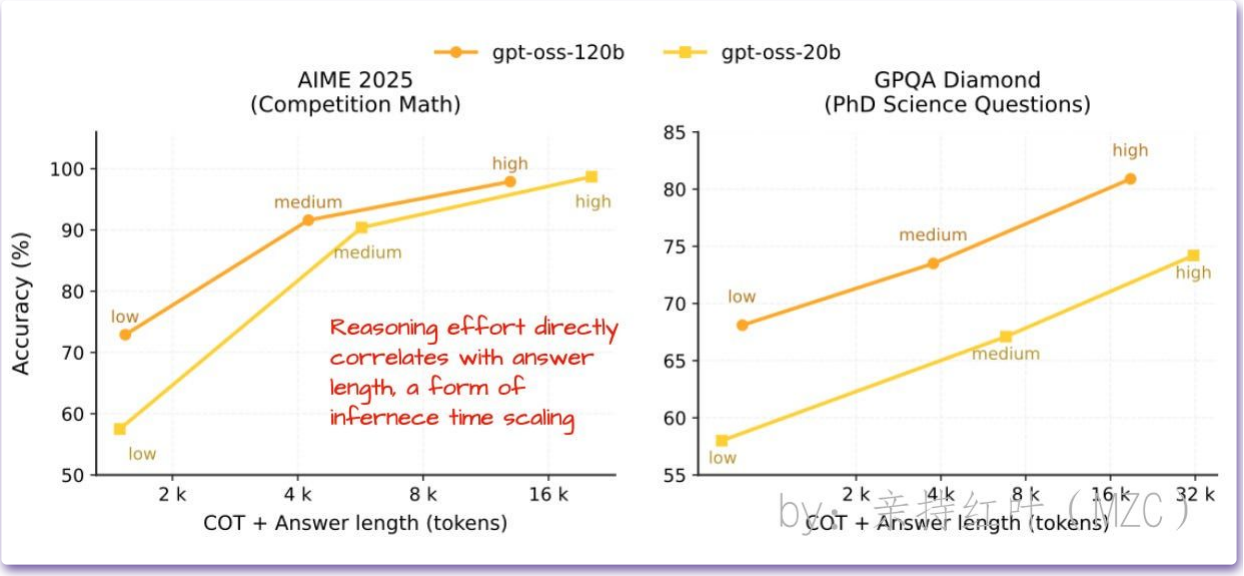
有意思的是，GPT-oss 的“训练时长”估算同时包含了两部分：用于指令跟随的监督学习训练，以及用于推理能力的强化学习训练；而 DeepSeek V3 只是一个预训练的基础模型，后

续的 DeepSeek R1 是在它之上单独训练出来的。

1.4.2 Reasoning Efforts

- 如上一节所述，GPT-OSS 模型是推理模型。然而特别有趣的是，它们的训练方式使得用户可以通过推理时间缩放轻松控制推理程度。

  具体来说，gpt-oss 模型可以接收“`Reasoning effort: low/medium/high`”指令作为其系统提示的一部分，这直接影响响应长度和准确性，如图下图所示。



  这种可调节的程度很有用，因为它能让我们在成本、算力和准确率之间取得平衡。比如，当任务很简单——像回答一个直接的知识问答或修正一个小拼写错误——我们就可以跳过扩展式的推理。这样既省时间和资源，也能避免不必要的长回复和冗长的推理过程。

  与 Qwen3 或 OLMo 不同，OpenAI 没有在基于强化学习的推理训练之前发布基础模型，这多少有些遗憾。基础模型对于研究推理方法的研究人员来说，是极其宝贵的起点（这也是我目前喜欢使用 Qwen3 Base 的原因之一）。我猜测，OpenAI 的决定更多是出于行业和生产用例的考虑，而非出于研究方面的考虑。

- 需要说明两点：

  早期的 Qwen3 原始模型也提供了一个开关，用来开启或关闭“思考（推理）”模式：在分词器里把 enable_thinking 设为 True/False，就会简单地给输入加上 标签，从而关闭或启用推理行为。

  不过，Qwen3 团队在最近几周更新了他们的模型，不再采用这种“混合式”的做法，而是转向分别提供独立的 Instruct / Thinking / Coder 版本。

  原因是混合模式与单个模型相比性能较低：

> After discussing with the community and reflecting on the matter, we have decided to abandon the hybrid thinking mode. We will now train the Instruct and Thinking models separately to achieve the best possible quality. **Source**

1.4.3 MXFP4 Optimization: A Small But Important Detail

- 有个有趣的意外是：OpenAI 发布的 gpt-oss 模型里，针对 MoE 专家部分采用了 MXFP4 量化方案。

  量化格式曾经是一个小众话题，主要与移动或嵌入式 AI 相关，但随着模型规模的扩大，这种情况发生了变化。在这种情况下，MXFP4 优化允许模型在单 GPU 设备上运行。

  实际情况如下：

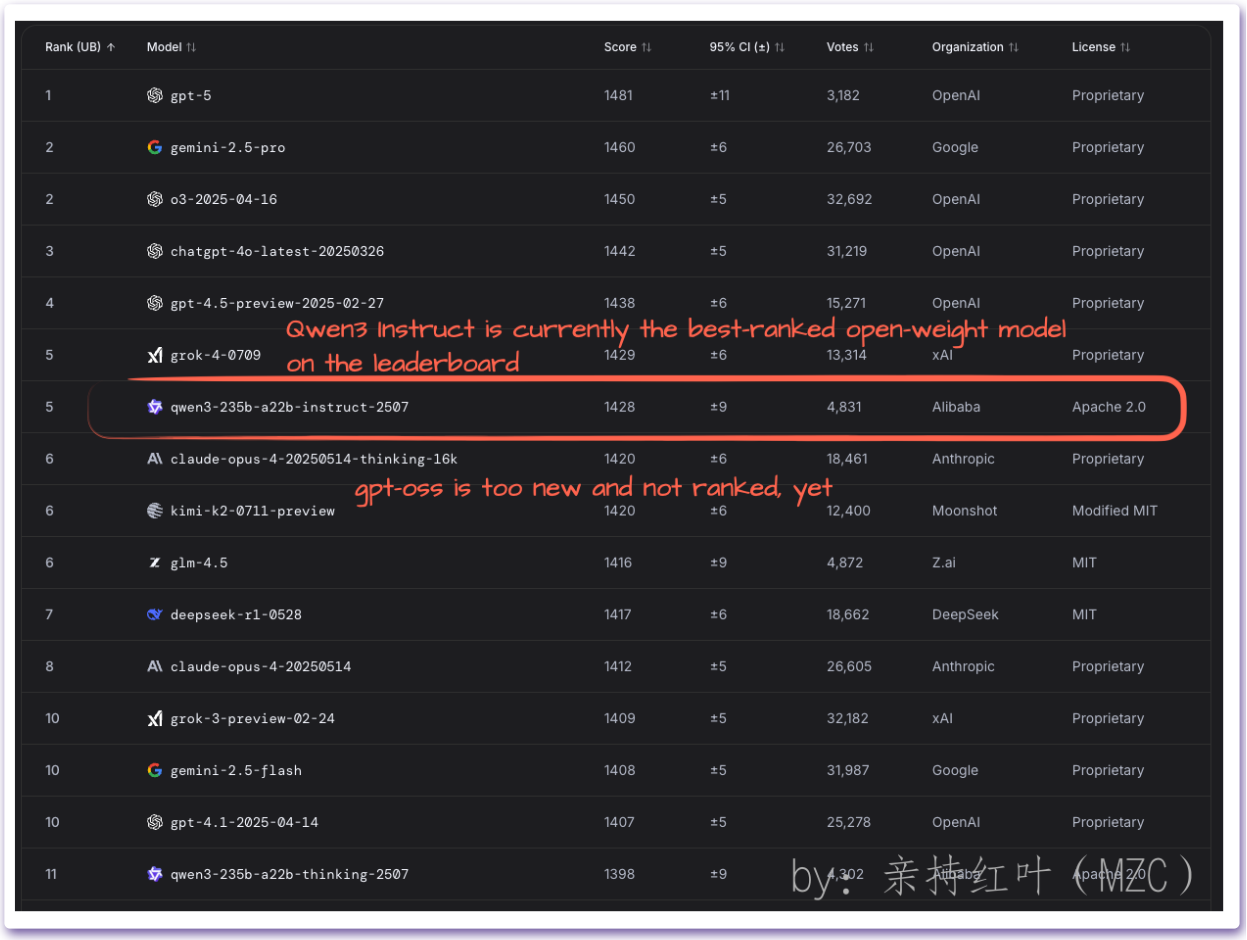
  1. 大型模型（例如 120B）可安装在单个 80GB H100 或更新的 GPU 上。虽然不是消费级硬件，但租用一台配备 1 个 H100 的机器比租用多台 H100 的机器便宜得多。此外，我们不必担心在 GPU 之间分配模型并增加通信开销。AMD MI300X 显卡从第一天起就支持，真是太好了！
  2. 较小的 20B 型号甚至可以装入 16 GB 的 VRAM；需要注意的是，它必须是 RTX 50 系列 GPU 或更新版本才能支持 MXFP4。

- 请注意，这些模型也可以在不支持 MXFP4 的旧硬件上运行，因此会消耗更多 RAM。如果没有 MXFP4 优化，bfloat16 模型的内存消耗会更大，例如 48 GB (gpt-oss-20b) 和 240 GB (gpt-oss-120b)。

  顺便说一句，我可以在 Mac Mini 上使用 ollama 轻松运行 gpt-oss-20b 模型。它占用大约 13.5 GB 内存，这确实很合理。

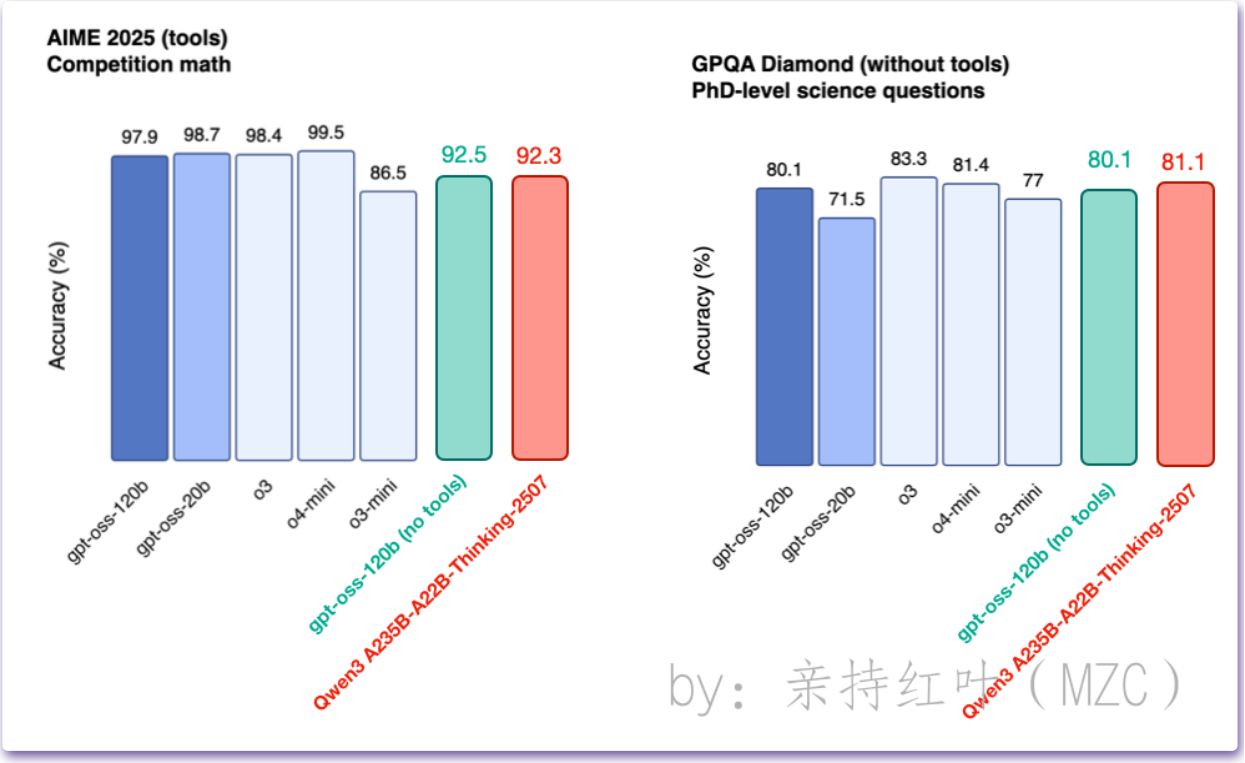
1.4.4 Benchmarks

- 这些模型对于独立基准测试来说仍然有点太新了。查看 **LM Arena 排行榜后**，我发现 gpt-oss 尚未上榜。因此，根据 LM Arena 用户的反馈，Qwen3-Instruct 目前仍然是最高权重的模型。



Rank (UB)	Model	Score	95% CI (±)	Votes	Organization	License
1	gpt-5	1481	±11	3,182	OpenAI	Proprietary
2	gemini-2.5-pro	1460	±6	26,703	Google	Proprietary
2	o3-2025-04-16	1450	±5	32,692	OpenAI	Proprietary
3	chatgpt-4o-latest-20250326	1442	±5	31,219	OpenAI	Proprietary
4	gpt-4.5-preview-2025-02-27	1438	±6	15,271	OpenAI	Proprietary
5	grok-4-0709	1429	±6	13,314	xAI	Proprietary
5	qwen3-235b-a22b-instruct-2507	1428	±9	4,831	Alibaba	Apache 2.0
6	claude-opus-4-20250514-thinking-16k	1420	±6	18,461	Anthropic	Proprietary
6	kimi-k2-0711-preview	1420	±6	12,400	Moonshot	Modified MIT
6	glm-4.5	1416	±9	4,872	Z.ai	MIT
7	deepseek-r1-0528	1417	±6	18,662	DeepSeek	MIT
8	claude-opus-4-20250514	1412	±5	26,605	Anthropic	Proprietary
10	grok-3-preview-02-24	1409	±5	32,182	xAI	Proprietary
10	gemini-2.5-flash	1408	±5	31,987	Google	Proprietary
10	gpt-4.1-2025-04-14	1407	±5	25,278	OpenAI	Proprietary
11	qwen3-235b-a22b-thinking-2507	1398	±9	4,302	Alibaba	Apache 2.0

查看 gpt-oss 公告帖中提供的推理基准，我们可以看到 gpt-ossmodels 与 OpenAI 的专有模型以及 Qwen3 相当。



不过，需要加上一点前提：gpt-oss-120b 的规模几乎只有 Qwen3 A235B-A22B-Thinking-2507 的一半，而且它可以在单块 GPU 上运行。

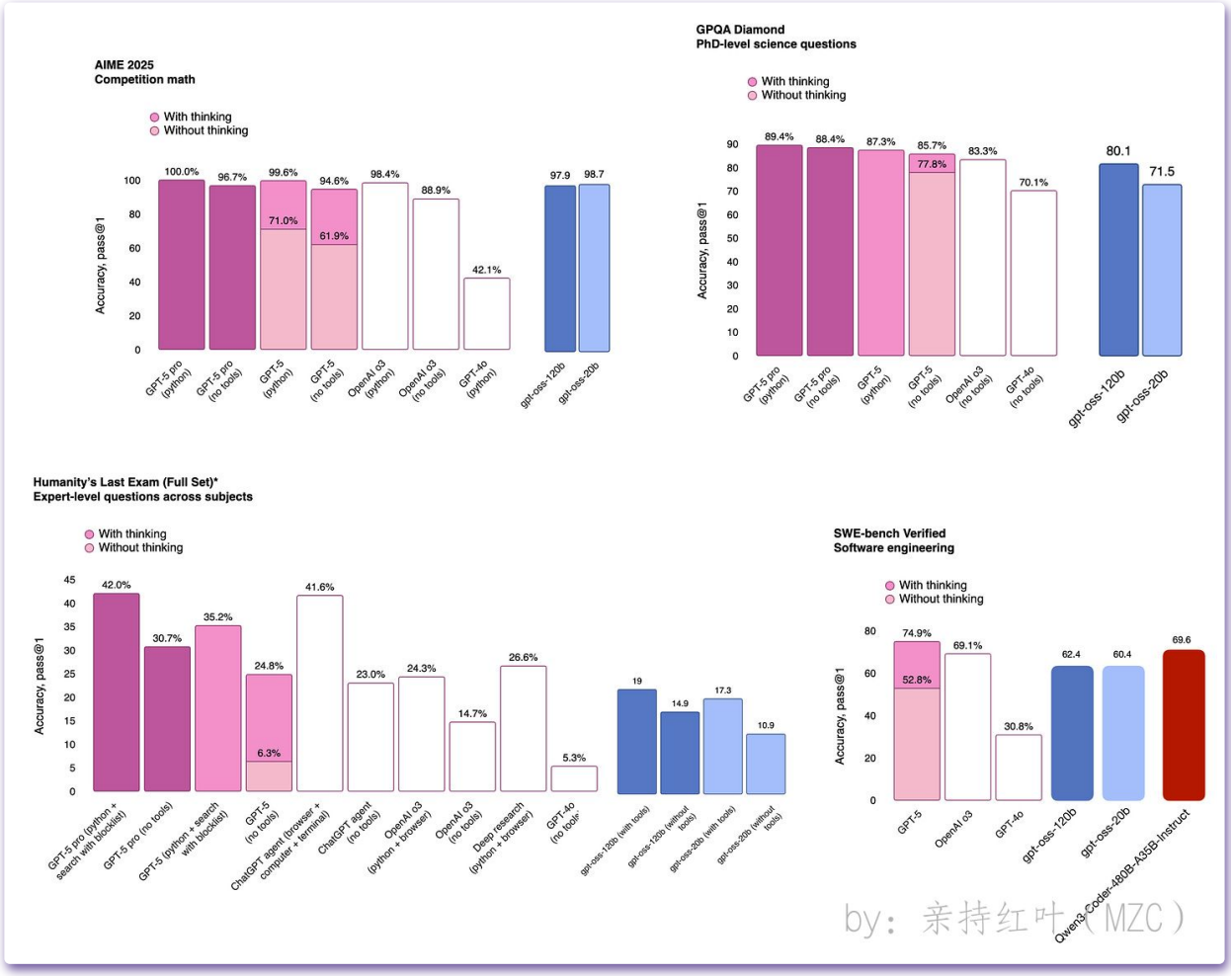
基准测试性能并不总是反映实际可用性。在过去几天有限的使用中，我发现 gpt-oss 相当强大。不过，正如其他人所观察到的，它似乎确实有较高的幻觉倾向（这一点在其模型卡中也提到了）。

这可能源于其训练侧重于数学、谜题和代码等推理任务，这可能导致一些“常识遗忘”。不过，由于 gpt-oss 在设计时就考虑到了工具的使用，因此这种限制随着时间的推移可能会逐渐减弱。开源 LLM 中的工具集成仍处于早期阶段，但随着其逐渐成熟，我预计我们会越来越多地让模型在回答事实或基于知识的查询时参考外部资源（例如搜索引擎）。

如果发生这种情况，那么优先考虑推理能力而非记忆力或许是明智之举。这很像人类在学校（或生活中）的学习，解决问题的能力通常比记忆事实更重要。

1.5 gpt-oss and GPT-5

- OpenAI 度过了忙碌的一周，在 gpt-oss 发布后不久，就发布了期待已久的 GPT-5 模型。GPT-5 的发布非常引人注目。如果说有什么要说的，那就是他们的开源模型在基准性能方面的表现，与其最佳产品相比，确实非常出色，这让我感到非常惊讶。



总而言之，尽管有些人认为该版本被过度炒作，但我很高兴我们拥有了一套真正强大的开放权重模型，它们与最好的专有模型并无太大差距。当然，基准测试通常不能准确反映实际使用情况，而且由于使用情况有限，现在下结论还为时过早。但我认为，对于喜欢使用开放权重和本地（或私人托管）模型的人来说，这是一个好时机。